

Python Codegolf Cheatsheet

Conditions

Inline conditions

```
if X:  
    print(A)  
else:  
    print(B)
```

40 characters

```
print(A if X else B)
```

20 characters

AND conditions

```
if X and Y:  
    print(A)  
else:  
    print(B)
```

46 characters

```
print([A,B][X and Y])
```

21 characters

And even better with **bitwise operators

```
print([A,B][X&Y])
```

17 characters!

Smart split

You can leave the space out before an operator if it is preceded by a number

```
print(5<X and Y<5)
```

18 characters

```
print(X<5and Y<5)
```

17 characters

Multiple comparisons

```
print(0<X and X<5)
```

18 characters

```
print(0<X<5)
```

12 characters

And even better:

```
print(X<5and Y<5)
```

17 characters

```
print(X<5>Y)
```

12 characters

But why stop here??

```
print(X>0and Z>0and Y<X)
```

24 characters

```
print(Z>0<X>Y)
```

14 characters!!

Multiple equalities

You can use tuples to compare multiples values

```
print(X==5 and Y==6 and Z==7)
```

30 characters

```
print((X,Y,Z)==(5,6,7))
```

23 characters

⚠ This does not save space for only two comparisons

Booleans

In Python, booleans are also integers

```
print([5,9][X])
```

15 characters

```
print(5+4*X)
```

12 characters

Difference of one

```
print([5,6][X])
```

15 characters


```
print(5+X)
```

10 characters

First num is one

```
print([1,6][X])
```

15 characters

```
print(6**X)
```

11 characters

Of course, this is Python so it also works with strings!

```
print(['hello', 'world'][X])
```

27 characters

```
print(X*'world'or'hello')
```

25 characters

Evil string indexing

The full slicing operator syntax is as follow:

```
array[start:stop:step]
```

The default values if ommitted are:

```
start = 0  
stop = -1  
step = 1
```

The step value can be very useful!

```
print(['YES', 'NO'][X])
```

22 characters

```
print('YNEOS'[X::2])
```

20 characters

And we can keep going:

```
print(['hello', 'world', 'fish'][X])
```

34 characters

```
print('hwfeoilrsllhod'[X::3])
```

29 characters

Output

```
print(', '.join(['A','B']))
```

27 characters

```
print(*'AB',sep=', ')
```

21 characters

To list the alphabet letters

```
import string  
print(string.ascii_lowercase)
```

44 characters

```
string=[chr(i+97)for i in range(26)]
```

36 characters

```
string='abcdefghijklmnopqrstuvwxyz'
```

35 characters

```
string=map(chr,range(97,123))
```

29 characters

Loops

Simple inline loops:

```
a=[]  
for i in range(10):  
    a.append(i)
```

42 characters

```
a=[i for i in range(10)]
```

24 characters

Always avoid nested loops!

```
for a in range(3):  
    for b in range(5):  
        print(a,b)
```

62 characters

```
for a in range(X*Y):print(A//X,a%X)
```

35 characters

You can even do it with three or more loops:

```
for i in range(5):  
    for j in range(6):  
        for k in range(7):  
            print(i,j,k)
```

96 characters

```
for k in range(5*6*7):  
    print(k//6//7, k%(6*7), k%7)
```

56 characters

And if you loop less than 4 times:

```
for a in range(4):func()
```

25 characters

```
for a in 0,1,2,3:func()
```

23 characters

This is also useful when you need `step != 1`

If the value of `a` doesn't matter:

```
for a in range(10):func()
```

25 characters

```
for _ in [1]*10:func()
```

22 characters

And the reason why interpreted languages are the best:

```
exec("foo();"*10)
```

17 characters

Variables

Functions

```
def c(a):  
    return a+1
```

25 characters

```
c=lambda a:a+1
```

14 characters

Iterables unpacking

Here you can replace `range(10)` with most iterables

```
A=list(range(10))
```

17 characters

```
*A,=range(10)
```

13 characters

Same thing with multiple variables:

```
l='abcdef'  
A=l[0]  
B=l[2]  
C=l[3:]
```

35 characters

```
A,_,B,*C='abcdef'
```

17 characters

This is also useful for long function names:

```
a,b,c=input(),input(),input()
```

29 characters

```
i=input;a,b,c=i(),i(),i()
```

25 characters

Or with eval():

```
a,b,c,d,e=input(),input(),input(),input(),input()
```

49 characters

```
a,b,c,d,e,_=eval("input(),"*5+'0')
```

34 characters

Iterables

Lists

Most "normal" list operations can be shorter:

```
# List creation
A,B=4,[] #8 chars
A,*B=4, #7 chars

# Getting first item
A=L[0] #6 Chars
A,*_=L #6 Chars

# Getting last item
A=L[-1] #7 Chars
*_,A=L #6 Chars

# Removing first item
L.pop(0) #8 Chars
L=L[1:] #7 Chars
_,*L=L #6 Chars
```

Appending an item

L.append(A) #11 Chars

L+= [A] #6 Chars

L+=A, #5 Chars

Extending a list

A.extend(B) #11 Chars

A+=B #4 Chars

Inserting items into a list

L.insert(i,A) #13 Chars

L[:i]+=A #8 Chars

Reversing a list

L=L.reverse() #13 Chars

L=L[::-1] #9 Chars

A very elegant way to iterate on a list in reverse using bitwise operators:

```
L[::-1][A]
```

10 characters

```
L[~A]
```

5 characters

Conversion of iterable to list

You might need this to call len() for example

```
print(len(list(A)))
```

19 characters

```
print(len(*A))
```

16 characters

Sets

Python sets have operators very similar to the bitwise operators:

```
setA|setB # Union  
setA&setB # Intersection  
setA-setB # Difference  
setA^setB # Symmetric difference (XOR)
```

There are also variants with = for in-place operations:

```
setA|=setB # Union  
setA&=setB # Intersection  
setA-=setB # Difference  
setA^=setB # Symmetric difference (XOR)
```

Here is how to check if an element is in a set:

```
var in setA
```

11 characters

```
{var}&setA
```

10 characters

Operators

Bitwise operators

`~n` is equivalent to `n+1`

`~-n` is equivalent to `n-1`

This can be combined with other operators to gain more characters

Boolean operators

Here are a few more operator alternatives

```
a and b  
a*b  
a&b
```

```
a or b  
a|b
```

Math operators

```
x=math.floor(A/B)
```

17 characters

```
X=A//B
```

6 characters

And same technique for ceil:

```
X=math.ceil(A/B)
```

16 characters

```
X=-(-A//B)
```

10 characters

And for square root

```
X=math.sqrt(A)
```

14 characters

```
X=A**.
```

7 characters

There are many more techniques to use, just make sure to practice!

Sources

<https://www.codingame.com/blog/code-golf-python/>

<https://www.geeksforgeeks.org/python/code-golfing-in-python/>